ServiceNow Development

Import Data using Transform Maps (Spreadsheet)

Project Documentation

1. Introduction

Project Title: Employee Data Import using Transform Maps (Spreadsheet) - ServiceNow

Team Members:

Name	Role
Member 1	Data Source and import set configuration
Member 2	Transform Map and field mapping
Member 3	Coalesce, scripts and department population
Member 4	ACLs, Impersonate User testing and documentation

2. Project Overview

Purpose: Employee data often arrives in spreadsheets (Excel/CSV). The goal of this project is to import that data into ServiceNow using Import Sets and Transform Maps, link each record to an employee, pull details such as department into the record for easier reporting, and secure the data using Access Control Lists (ACLs) tested with Impersonate User.

Features:

- Bulk import of Excel/CSV data through a Data Source and a staging (Import Set) table.
- Transform Map with Field Maps from staging table to target table.
- Employee ID mapped to the Employee (sys_user) reference field; Coalesce prevents duplicates.
- Department and other employee details pulled into each record (field map script).
- Transform History and Import Log for verification.
- ACLs on the target table and fields, tested using Impersonate User.
- Optional scheduled imports and department-wise reports.

3. Architecture

Frontend (ServiceNow UI): The project does not use React. The user interface is the standard ServiceNow Next Experience web UI: the Data Sources, Import Sets and Transform Maps modules for the administrator, and list views, forms and reports for viewing the imported records.

Backend (Platform / Server-side): Node.js and Express.js are not used. Processing runs on the ServiceNow instance: the Import Set engine loads the file into the staging table, the Transform Map engine applies the Field Maps, and server-side JavaScript (field map scripts using GlideRecord) transforms values. Optional access is through the REST Import Set API.

Database (ServiceNow tables): MongoDB is not used. Data is stored in ServiceNow tables on the instance database:

Table	Purpose	Key fields
Staging table (u_employee_import)	Holds raw rows loaded from the spreadsheet	u_first_name, u_last_name, u_employee_id, u_email, u_department
Target table (employee-linked records)	Final records created by the Transform Map	Employee (reference to sys_user), Department, Email and other mapped fields
Employee (sys_user)	Source of employee details; referenced by the target table	User ID, name, email, department
Transform History / Import Log	Results of each transform run	Inserted, updated, ignored counts; error messages

4. Setup Instructions

Prerequisites:

- A ServiceNow instance (for example a free Personal Developer Instance from developer.servicenow.com).
- Admin access (admin role, or import_admin / import_transformer roles).
- Microsoft Excel or any CSV editor, and a web browser (Chrome / Edge).
- The employee spreadsheet (.xlsx or .csv) with a header row.

Installation: No code needs to be cloned or installed; the project is configured inside the instance. No environment variables are needed unless the REST API is used (instance URL and credentials).

1. Sign in to developer.servicenow.com and request a Personal Developer Instance, then log in as admin.
2. Prepare the spreadsheet: first row as column headings, no blank rows or merged cells.
3. Create the target table (or choose an existing table) with a reference field to Employee (sys_user) and a Department field.
4. Create the Data Source (System Import Sets > Administration > Data Sources) and attach the file.
5. Create the Transform Map and Field Maps, then set Coalesce on Employee ID.

5. Folder Structure

ServiceNow is configured on the platform rather than as client/server code folders, so the structure is described by the application artefacts and the project repository.

Client (user-facing artefacts):

- Application menu and modules for the target table.
- List views and forms showing Employee and Department.
- Reports and dashboards grouped by department.

Server (import and security artefacts):

- Data Source (file type, format, header row, sheet number).
- Staging table (u_) and Import Set records.
- Transform Map, Field Maps and scripts (field map scripts, onBefore / onAfter).
- ACL records for the target table and fields; roles.

- Scheduled Import (optional).

Suggested project repository layout:

```
project-root/
  dataset/        sample employee spreadsheet (.xlsx / .csv)
  scripts/        field map and transform scripts
  update-set/     exported update set (XML)
  screenshots/    output screenshots
  docs/           project documentation and reports
  README.md
```

6. Running the Application

There are no frontend or backend servers to start with npm; the instance is always running. To run the import:

1. Open the Data Source and click **Load All Records** (this creates an Import Set).
2. Open the Transform Map and click **Transform**, then select the Import Set.
3. Open **Transform History** and the target table to verify the records.

7. API Documentation

The import is normally run through the UI. The ServiceNow REST Import Set API can optionally post rows into the staging table, and the Table API can read the results. The examples below are illustrative; replace the instance name and table names with yours.

Method	Endpoint	Description	Parameters / body
POST	/api/now/import/{stagingTableName}	Insert a row into the staging table; the Transform Map then runs	JSON body with staging field values
GET	/api/now/table/{targetTableName}	List records from the target table	sysparm_query, sysparm_limit, sysparm_display_value

Example request:

```
POST https://<instance>.service-now.com/api/now/import/u_employee_import
Content-Type: application/json

{
  "u_first_name": "Arun",
  "u_last_name": "Kumar",
  "u_employee_id": "arun.k",
  "u_email": "arun@example.com",
  "u_department": "IT"
}
```

Example response (illustrative; the structure depends on the instance):

```
{
  "import_set": "ISET0010001",
  "staging_table": "u_employee_import",
  "result": [
    {
      "transform_map": "Employee Import Transform",
      "table": "<target table>",
      "status": "inserted",
      "sys_id": "<record sys_id>"
    }
  ]
}
```

8. Authentication

The project does not use JWT tokens. Authentication and authorisation are handled by the ServiceNow platform:

- **Authentication:** users sign in to the instance with their credentials (or SSO if configured); REST calls use basic authentication or OAuth.
- **Roles:** import-related work needs admin or import_admin / import_transformer roles; a custom role (for example HR manager) can be created for viewing employee-linked records.
- **Authorisation (ACLs):** Access Control Lists on the target table and fields define which roles can read, write, create or delete records and fields.
- **Session testing:** the Impersonate User feature lets an administrator act as another user to confirm that ACLs behave correctly, then return to their own session.

9. User Interface

Insert screenshots showing the main UI features of the project.

Figure 1: Data Source with the spreadsheet attached

Figure 2: Transform Map and Field Maps

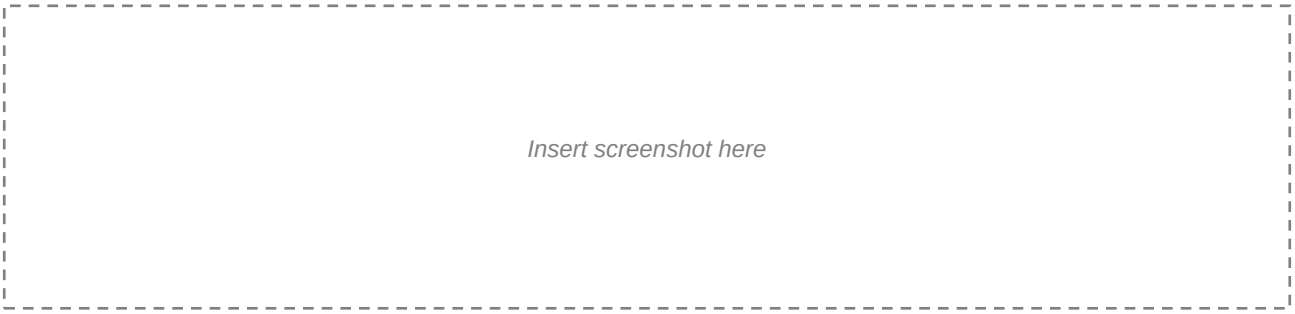

Figure 3: Target table list view with Employee and Department

10. Testing

Strategy: manual functional testing on the instance using a small sample file first, then a larger file for performance. Verification uses Transform History, the Import Log and Impersonate User. The ServiceNow Automated Test Framework (ATF) can be added for repeatable tests.

ID	Test case	Expected result	Status
T-1	Load file into staging table	All rows visible in the Import Set table	
T-2	Run Transform Map	Records inserted; counts shown in Transform History	
T-3	Employee link and department	Records linked to the employee with department shown	
T-4	Re-import the same file (Coalesce)	Records updated; no duplicates	
T-5	Blank / unknown Employee ID	Row ignored or flagged in the Import Log	
T-6	Impersonate an unauthorised user	Restricted fields and records hidden or read-only	
T-7	Impersonate an authorised user	Permitted records and fields visible	

11. Screenshots or Demo

Insert the final output screenshots and the demo link below.

Insert screenshot here

Figure 4: Transform History showing inserted / updated counts

Insert screenshot here

Figure 5: ACL record and Impersonate User test result

Demo link: _____

12. Known Issues

- Reference fields stay empty if the Employee ID in the file does not exactly match a user record.
- Date and choice values must be in a consistent format or they fail to import.
- Merged cells, blank rows or wrong header and sheet settings can cause no rows or wrong rows to load.
- Very large files may need to be split into batches.
- A misconfigured ACL can block valid users, so test it with Impersonate User.

- A Personal Developer Instance may hibernate after a period of inactivity and need to be woken up.

13. Future Enhancements

- Schedule imports to keep employee data up to date automatically.
- Integrate with an HR system or LDAP instead of spreadsheets.
- Add validation scripts and notifications for failed rows.
- Extend the same approach to other tables (assets, incidents, departments).
- Build department-wise dashboards and add more detailed role-based ACLs.